Compiladores – Folha de revisão

Gonçalo Sousa Aulas 1–6. As receitas do resumo, sem exemplos: para rever antes do exame.

Análise lexical

Expressões regulares. a, ε, $M \mid N$, MN, M^*. Prioridade: * > concatenação > $\mid$ ($ab|c = (ab)|c$). Abreviaturas: `M?` $= M \mid \varepsilon$, `M+` $= MM^*$, `[abc]`, `[a-z]`, `[^a-z]`, `.` (tudo menos `\n`), `"a+"` (literal).

Parar no primeiro terminador: proibir o terminador no meio, ex.: `"<!--" ([^-]|"-"[^-])* "-->"`. Nunca `"/*" .* "*/"` (apanha do primeiro $/*$ ao último $*/$).

Thompson (ER → NFA), um fragmento com uma entrada e uma saída por sub-expressão:

	Construção
σ	entrada $\xrightarrow{\sigma}$ saída
AB	saída de A ligada à entrada de B
$A \mid B$	entrada nova $\xrightarrow{\varepsilon} A$ e B; saídas de A e $B \xrightarrow{\varepsilon}$ saída nova
A^*	entrada nova $\xrightarrow{\varepsilon} A$ e $\xrightarrow{\varepsilon}$ saída nova; saída de $A \xrightarrow{\varepsilon}$ entrada de A e saída nova

Subconjuntos (NFA → DFA).

1. $S_0 = \text{closure}(\{q_0\})$ (tudo o que se alcança só por ε).
2. Para cada estado S ainda não tratado e cada símbolo σ: $\text{move}(S, \sigma)$ = destinos das transições σ que saem de estados de S; depois $\delta'(S, \sigma) = \text{closure(move)}$. Vazio: sem transição. Conjunto novo: estado novo.
3. Finais: os conjuntos que contêm um final do NFA.

Minimização: estados com as mesmas transições para todos os símbolos e o mesmo estatuto (final ou não) fundem-se.

Geradores (Alex/Flex): *longest match* primeiro, depois *first match* → palavras reservadas **antes** da regra de identificadores. Alex: `padrão { \s -> Token }`, `alexScanTokens`. Flex: `padrão { ação C }`, `yytext`, `yylex()`.

Gramáticas

$G = (\Sigma, N, S, P)$, produções $X \to \alpha$. $L(G) = \{w \in \Sigma^* : S \Rightarrow^* w\}$. Derivação **mais à esquerda** (LL) / **mais à direita** (LR, ao contrário). **Ambígua** = uma palavra com **duas árvores** diferentes (duas derivações não chegam).

ER → gramática: $rs \to$ um lado direito $r\,s$; $r \mid s \to X \to r \mid s$; $r^* \to X \to rX \mid \varepsilon$; $r^+ \to X \to rX \mid r$.

Desambiguar expressões: $E \to E + T \mid T$, $T \to T * F \mid F$, $F \to$ `num` $\mid (E)$. Recursão à esquerda → associatividade à esquerda; um nível por precedência (o mais "fundo" liga mais).

Dangling else: o `else` liga ao `if` mais próximo. Gramática com M (*matched*) / U (*unmatched*), ou deixar e fazer *shift* no conflito.

Análise LR

Ações: *shift* (empilha o terminal), *reduce* $X \to \gamma$ (tira γ do topo, empilha X). Gramática aumentada $S' \to S\,\$$.

Tabela: $s\,q$, $r\,k$, $g\,q$ (só em não-terminais), a, vazia = erro. **Reduce** k ($X \to \alpha_1..\alpha_n$): tira n estados, olha para o estado do topo, empilha o g da coluna X desse estado.

Items LR(0): $A \to \beta \bullet \gamma$ (β está no topo). Inicial $A \to \bullet\gamma$; completo $A \to \gamma\bullet$. Comprimento n dá $n + 1$ items; $A \to \varepsilon$ dá só $A \to \bullet$.

Autómato: estado inicial fecho($\{S' \to \bullet S\,\$\}$).

- **fecho(I)**: para cada item com o ponto antes de um não-terminal B, junta todos os $B \to \bullet\beta$; repete.
- **goto(I, X)**: items de I com o ponto antes de X, avança o ponto, aplica o fecho.

Tabela LR(0): terminal $i \xrightarrow{t} j \to s\,j$; não-terminal $i \xrightarrow{X} j \to g\,j$; item completo da produção $k \to r\,k$ em **todas** as colunas terminais; $S' \to S \bullet \$ \to a$ na coluna \$. Uma entrada com duas ações = **conflito** (shift/reduce ou reduce/reduce).

NULLABLE, FIRST e FOLLOW. $\mathrm{N}(\varepsilon) = \text{True}$, $\mathrm{N}(a) = \text{False}$, $\mathrm{N}(\alpha\beta) = \mathrm{N}(\alpha) \wedge \mathrm{N}(\beta)$, $\mathrm{N}(X) = \bigvee \mathrm{N}(\gamma_i)$. $\mathrm{FIRST}(\varepsilon) = \varnothing$, $\mathrm{FIRST}(a) = \{a\}$, $\mathrm{FIRST}(\alpha\beta) = \mathrm{FIRST}(\alpha) \cup (\mathrm{FIRST}(\beta)$ se $\mathrm{N}(\alpha))$, $\mathrm{FIRST}(X) = \bigcup \mathrm{FIRST}(\gamma_i)$. FOLLOW: $\$ \in \mathrm{FOLLOW}(S)$; para cada $Y \to \alpha X \beta$: $\mathrm{FIRST}(\beta) \subseteq \mathrm{FOLLOW}(X)$; se $\mathrm{N}(\beta)$ (β vazio incluído), $\mathrm{FOLLOW}(Y) \subseteq \mathrm{FOLLOW}(X)$. Tudo começa em False/$\varnothing$; NULLABLE → FIRST → FOLLOW; repetir até estabilizar.

SLR(1): igual à LR(0), mas o *reduce* de $A \to \gamma\bullet$ só nas colunas de $\mathrm{FOLLOW}(A)$.

LR(1): items $(A \to \alpha \bullet \beta,\ a)$; a transição-$\varepsilon$ de $(A \to \alpha \bullet B\gamma, a)$ vai para $(B \to \bullet\beta, b)$ com $b \in \mathrm{FIRST}(\gamma a)$; *reduce* só com o *look-ahead* a. **LALR(1)** = LR(1) com os estados de items LR(0) iguais fundidos (é o que os geradores usam).

Gramática ambígua ⇒ conflitos sempre. Conflitos ⇏ ambígua.

Geradores LR (Happy / Bison)

	Happy	Bison
Ação	expressão: `{ $1 + $3 }`	bloco: `{ $$ = $1 + $3; }`
Valor de um terminal	`$$` no padrão de `%token`	o *lexer* põe em `yylval`
Tipos diferentes	inferidos	`%union` + `%token <num>` + `%type <exp>`
Função	`%name parser`, recebe `[Token]`	`yyparse()`, chama `yylex()`
Relatório	`happy -i` → `.info`	`bison --report=all` → `.output`

`$1`, `$2`, … = valores do lado direito por ordem. A ação corre no **reduce**, de baixo para cima na árvore. Terminais de um carácter (`'+'`) usam o código ASCII (Flex: `return yytext[0];`).

Conflitos: por omissão, **shift** (tudo associa à direita, com a mesma precedência). Declarações antes do `%%`, **cada linha com mais precedência do que as anteriores**: `%left`, `%right`, `%nonassoc`.

Regra: a precedência da regra R é a do seu **último terminal**; *token* a com mais precedência → *shift*; menos → *reduce*; igual → `%left` *reduce*, `%right` *shift*, `%nonassoc` erro.

Sintaxe abstrata

Concreta: um nó por símbolo da derivação. **AST**: um nó por operação, sem parêntesis, palavras reservadas nem não-terminais de desambiguação; as ações semânticas constroem-na logo.

	Representação	Distinguir casos
Haskell	`data` com um construtor por alternativa	encaixe de padrões
C	`struct` com *tag* (`enum`) + `union`; funções `mk_...`	`switch` na *tag*
Java	classe abstrata + uma subclasse por alternativa	`instanceof` + *cast* (ou *visitor*)

Preferir a abordagem funcional: o compilador avisa se faltar um caso.

Receita em C/Flex/Bison (lab. ASTs): um `struct` com *tag* por categoria (`Expr`, `BoolExpr`, `Cmd`); um campo por tipo no `%union` e `%type <campo>` por não-terminal; a ação chama o construtor; a raiz vai para uma global (`root`), **definida** num `.c`; as funções sobre a AST (`eval`, `print...`) são recursivas, com um `switch` na *tag*. Identificadores: `yylval.s = strdup(yytext)` **no Flex**. Palavras reservadas antes do `ID` (*first match*). `c1 ; c2`, `if` e `else` pendente resolvem-se com `%left SEMI` < `%nonassoc THEN WHILE` < `%nonassoc ELSE`.

Análise LL (*top-down*)

Descida recursiva: uma função por não-terminal, um caso por produção, escolhida pelo próximo *token*; terminal → `consume`, não-terminal → chamada. Haskell: `[Token] -> [Token]` devolve o resto; `accepted = null . parseS`. C: `next` global, `advance()`, `consume(t)`; no fim `consume(EOF)` (ou $S' \to S\,\$$).

Recursão à esquerda: $A \to A\alpha \mid \beta \Rightarrow A \to \beta A'$, $A' \to \alpha A' \mid \varepsilon$. Expressões: $E \to TE'$, $E' \to +TE' \mid -TE' \mid \varepsilon$, $T \to FT'$, $T' \to *FT' \mid /FT' \mid \varepsilon$, $F \to$ `num` $\mid (E)$. **Fatorização**: $A \to \alpha\beta_1 \mid \alpha\beta_2 \Rightarrow A \to \alpha A'$, $A' \to \beta_1 \mid \beta_2$. Recursão à esquerda ⇒ não é LL(1).

Tabela LL(1): $X \to \gamma$ na linha X, colunas FIRST(γ); se N(γ), também colunas FOLLOW(X). LL(1) ⇔ no máximo uma produção por entrada; vazia = erro.

Pilha explícita: começa com S'. Topo terminal → tem de ser igual ao próximo: consumir e tirar. Topo não-terminal X → produção de table[X, próximo]: tira X, empilha o lado direito (1.º símbolo no topo). Pilha e entrada vazias → aceita.